

Scrubless — Architecture

Semantic video search · *Stop scrubbing. Start searching.*

Ernest Opara

2026-05-22

Contents

1 Overview	2
1.1 Design principles	2
2 Technology stack	4
3 How it works	5
3.1 Indexing pipeline	5
3.2 Search pipeline	5
4 Data model	7
5 API reference	8
6 Accounts, tiers & billing	9
6.1 Configuration (environment variables)	9
7 Deployment	10
7.1 Project layout	10
8 Roadmap — V2 (weekend of 2026-05-23)	11
9 Known limitations (V1)	13
10 About this document	14
11 Change log	14

1 Overview

Scrubless is a semantic video-search product. The whole user story is one sentence:

Upload a video → index it → search it in natural language → click a result and the player jumps to that exact moment.

Every uploaded video is split into still frames and an audio transcript. Each frame becomes a **CLIP embedding** — a vector capturing what the frame *looks like* — stored in a vector database alongside the transcript text for that moment. A search query is embedded the same way, and the database returns the frames whose meaning is closest to the query.

The governing constraint (from CLAUDE.md) is **radical simplicity**: one language (Python), one backend file (`app.py`), one frontend file (`index.html`), no Docker-for-dev, no microservices, no build step.

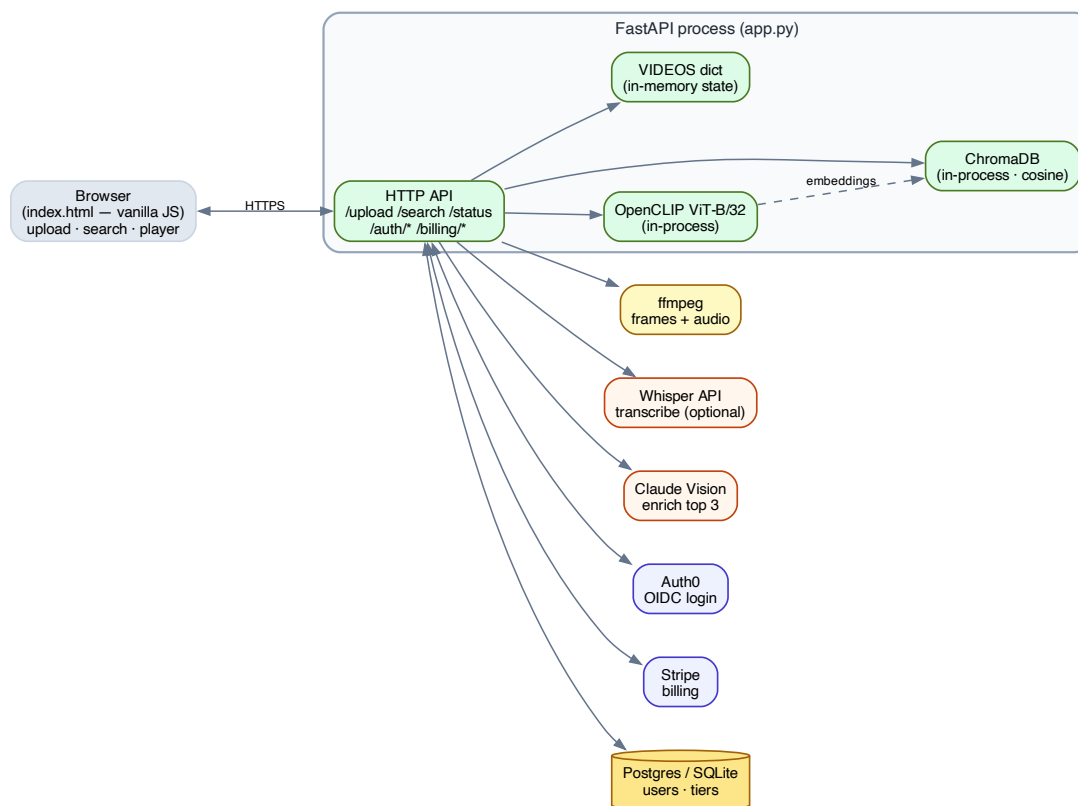


Figure 1: System architecture — one FastAPI process with the model and vector DB in-process; everything else is an external call.

1.1 Design principles

- **One file per layer.** All backend logic in `app.py`; all UI in `index.html`.

- **In-process everything.** CLIP and the vector DB run inside the FastAPI process — no sidecars, no network hops.
- **Graceful degradation.** Every external dependency (Whisper, Claude, Auth0, Stripe, Postgres) is optional; a missing key disables that feature instead of crashing the app.
- **Cheap to run.** Fits on one small Railway service / a single VPS.

2 Technology stack

Concern	Choice
Web framework	FastAPI (single file: <code>app.py</code>)
ASGI server	uvicorn
Video processing	ffmpeg via subprocess (frames + audio)
Visual embeddings	OpenCLIP ViT-B/32 (laion2b_s34b_b79k)
Vector search	ChromaDB (in-process <code>PersistentClient</code> , cosine)
Transcription	OpenAI Whisper API (<code>whisper-1</code>) — optional
Result enrichment	Anthropic Claude Vision (<code>claude-sonnet-4-6</code>)
Frontend	One <code>index.html</code> — vanilla JS + CSS, no build step
Auth	Auth0 (OIDC) via Authlib + signed session cookies
Accounts DB	SQLAlchemy 2.0 — SQLite (dev) / Postgres (prod)
Billing	Stripe Checkout + Customer Portal + webhooks
Deployment	Docker image on Railway; custom domain via GoDaddy
Docs	Markdown + Graphviz, rendered to PDF by pandoc

Deliberately *not* used: Go, Docker Compose for dev, Terraform/AWS/ECS/S3, React build pipeline, face recognition, knowledge graphs, YouTube video downloads, clip export (deferred or rejected per `CLAUDE.md`).

3 How it works

The FastAPI process is the entire backend. The only stateful stores are **ChromaDB** on disk (frame embeddings + metadata), the **relational DB** (the users table), and the in-memory **VIDEOS** dict (per-video processing status, lost on restart — acceptable for V1).

3.1 Indexing pipeline

When a video is uploaded, `process_video()` runs in a background thread.

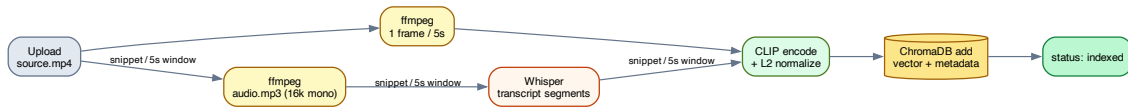


Figure 2: Indexing — frames and audio are extracted in parallel, joined per 5-second window, embedded, and written to ChromaDB.

1. **Frames** — ffmpeg at fps=1/5: one JPEG every **5 s** (FRAME_INTERVAL).
2. **Audio** — ffmpeg to 16 kHz mono MP3, small enough for Whisper. Wrapped in try/except: a video with no audio indexes visually and continues.
3. **Transcript** — Whisper (verbose_json) → [{start, end, text}]. Skipped silently without OPENAI_API_KEY.
4. **Embed** — each frame is CLIP-encoded, L2-normalized, and paired with the transcript overlapping its 5-second window.
5. **Store** — one segments.add() call: id = "{video_id}-{i}", the vector, metadata {video_id, timestamp, frame_path, transcript_segment}.
6. **Status** — progress updates per frame; status → indexed (or error).

3.2 Search pipeline

POST /api/search/{video_id} embeds the query into the same CLIP space and asks ChromaDB for the nearest frames.



Figure 3: Search — the query is embedded, matched against frame vectors, and the top 3 are enriched with a Claude Vision description.

1. Reject if the video isn't indexed (409) or the query is empty (400).
2. **Embed** the query (embed_text).
3. **Query** ChromaDB for the top **10**, filtered to the one video_id, ranked by cosine distance.

4. Convert distance to a **score** ($1.0 - \text{dist}$); attach frame URL, timestamp, transcript snippet.
5. **Enrich** the top **3** (ENRICH_TOP_N) with a 1–2 sentence Claude Vision description (best-effort; skipped without an Anthropic key).
6. Return the list; the UI renders clickable thumbnails that seek the player.

4 Data model

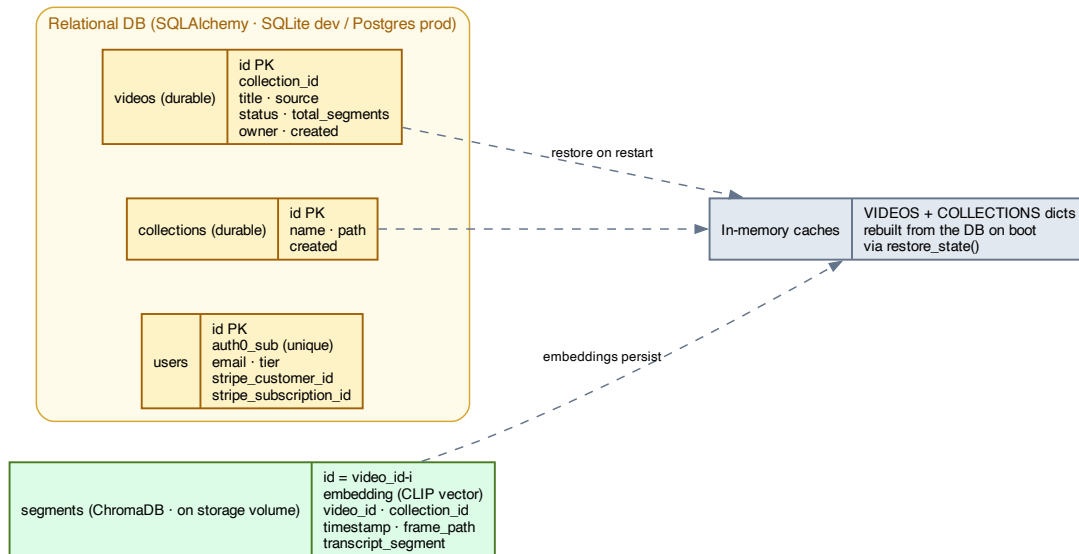


Figure 4: The relational DB holds durable videos + collections alongside users; ChromaDB holds the embeddings on the storage volume; the in-memory dicts are caches rebuilt on boot.

- **users / videos / collections** (SQLAlchemy) — the relational store. `users` holds accounts/tiers/Stripe IDs; `videos` and `collections` (V2) hold the **durable** metadata (title, source, status, owner, membership) so the library survives a restart. `DATABASE_URL` picks the backend: SQLite in dev, Postgres in prod (`postgres://` normalized to `postgresql://`).
- **segments** (ChromaDB, on the storage volume) — one row per frame: embedding + metadata (`video_id`, `collection_id`, `timestamp`, `frame_path`, `transcript_segment`). Distance space: **cosine**.
- **VIDEOS / COLLECTIONS** (in-memory) — fast caches of per-video status and folder membership, rebuilt from the DB on boot by `restore_state()` (any video left mid-index is resumed). No longer the source of truth.

5 API reference

Method	Path	Purpose
GET	/	Serve <code>index.html</code>
POST	/api/upload	Upload a video, start indexing (cap-gated)
GET	/api/status/{id}	Poll indexing status / progress
POST	/api/search/{id}	Natural-language search of one video
DELETE	/api/videos/{id}	Delete a video (sample is protected)
GET	/api/videos/{id}/source	Stream a video file, range-seekable (V2)
POST	/api/library/pick	Native folder chooser (local macOS) (V2)
POST	/api/library/scan	Index every video under a folder (V2)
POST	/api/library/create	Create an empty collection (upload) (V2)
POST	/api/library/{id}/upload	Upload a video into a collection (V2)
GET	/api/library/{id}	Collection status + per-video progress (V2)
POST	/api/library/{id}/search	Search across a whole collection (V2)
GET	/api/auth/login	Redirect to Auth0 Universal Login
GET	/api/auth/callback	OIDC callback → create/find user, session
GET	/api/auth/logout	Clear session + Auth0 logout
GET	/api/auth/me	Current user, upload cap, feature flags
POST	/api/billing/checkout	Create a Stripe Checkout session
POST	/api/billing/portal	Open the Stripe Customer Portal
POST	/api/billing/webhook	Stripe webhook → update tier
—	/storage/*, /scrubby/*	Static mounts (frames/video, mascot)

6 Accounts, tiers & billing

Access is gated by upload size; anonymous visitors are treated as the free tier.

Tier	Upload cap	Price	Account required
Free	500 MB	\$0	No (anonymous)
Pro	2 GB	\$10 / mo	Yes
Studio	10 GB	\$30 / mo	Yes

- **Caps** are enforced twice in `/api/upload`: a fast Content-Length reject, then a streaming backstop that deletes the partial file if the body overruns.
- **Free-tier hygiene** — anonymous uploads auto-expire after **24 h** (`expiry_sweep`, every 30 min). The sample and account-owned videos never expire.
- **Login** is Auth0 OIDC via Authlib; the session is a signed cookie. New subjects create a User row on first callback.
- **Billing** is Stripe hosted Checkout (subscription mode). Tier changes are driven by signature-verified `customer.subscription.*` webhooks, keyed by Stripe customer id. The Customer Portal handles upgrades/cancellation.

6.1 Configuration (environment variables)

Variable	Enables / controls
OPENAI_API_KEY	Whisper transcription (else visual-only)
ANTHROPIC_API_KEY	Claude Vision result enrichment
SESSION_SECRET / AUTH0_SECRET	Signed session-cookie key
APP_BASE_URL	Public base URL for OAuth/Stripe redirects
AUTH0_DOMAIN	Auth0 tenant (scheme/slash tolerated)
AUTH0_CLIENT_ID / _SECRET	Auth0 application credentials
DATABASE_URL	SQLite (dev) or Postgres (prod)
STRIPE_SECRET_KEY	Enables billing
STRIPE_WEBHOOK_SECRET	Verifies webhook signatures
STRIPE_PRICE_PRO / _STUDIO	Stripe price ids per plan

Feature flags derived at startup: `AUTH0_ENABLED` (all three Auth0 vars present), `STRIPE_ENABLED` (`STRIPE_SECRET_KEY` present). Missing keys downgrade the feature, never crash the app.

7 Deployment

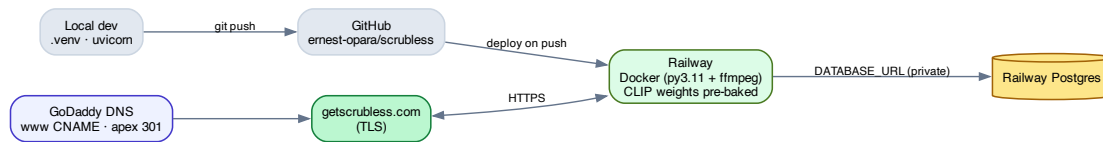


Figure 5: Deployment — push to GitHub triggers a Railway Docker build; the app reads Postgres over the private URL; GoDaddy DNS points the domain at Railway.

- **Container** — Dockerfile on `python:3.11-slim`, installs `ffmpeg`, `pip`-installs `requirements.txt`, pre-bakes CLIP weights, exposes 8080, runs `uvicorn`.
- **Host** — Railway (one service) via `railway.toml`. `.dockerignore` keeps `storage/`, `.env`, `.venv/`, `legacy/` out of the image.
- **Domain** — `getscrubless.com` via GoDaddy: `www CNAME` → Railway, `apex 301-forwarded`, `_railway-verify.www TXT`. TLS issued by Railway.
- **Durable storage (V2)** — Railway’s container filesystem is ephemeral, so the library survives redeploy only with **both**:
 1. a **persistent volume mounted at `/app/storage`** — holds uploaded videos, extracted frames, and the ChromaDB embeddings; and
 2. **Postgres** via `DATABASE_URL = ${Postgres.DATABASE_URL}` — holds the users / videos / collections tables.

Without both, uploads vanish on the next deploy. The sample is re-indexed each boot regardless (`ingest_sample`). On startup `restore_state()` rebuilds the in-memory caches from the DB and resumes any interrupted indexing.

7.1 Project layout

```
clipfind/
├── app.py                # entire backend
├── index.html            # entire frontend
├── requirements.txt
├── Dockerfile, railway.toml, .dockerignore
├── assets/sample.mp4     # auto-indexed demo clip (Big Buck Bunny)
├── scrubby/             # mascot SVGs + favicon
├── docs/
│   ├── ARCHITECTURE.md   # source of truth (this document)
│   ├── ARCHITECTURE.pdf  # rendered output
│   ├── render.sh         # diagrams + pandoc build
│   └── diagrams/*.dot    # Graphviz sources
├── storage/              # runtime: uploads, frames, chroma (gitignored)
└── legacy/               # archived Go + React + sidecar stack
```

8 Roadmap — V2 (weekend of 2026-05-23)

Theme: from demo → product. V1 is shipped, deployed, and monetized; V2 makes it durable and sticky.

Foundation:

1. **Durable storage + state — implemented & verified in production.** videos/collections metadata is persisted to Postgres and rebuilt on boot (`restore_state`), and files + embeddings sit on a persistent volume (`/app/storage`). Verified on `getscrubless.com` across a real redeploy: metadata, embeddings, frames, and source files all survived with no re-upload.
2. **Per-user library — pending.** The data is now durable; the UI for a logged-in user to browse/re-search their own videos is the next step. Anonymous uploads still auto-expire (24 h).

Marquee: Library Mode — search across a whole folder. Turn Scrubless from a single-clip tool into a search engine for a video *library*: one query returns the best moments across **every** video in a directory, each result naming its source file + timestamp.

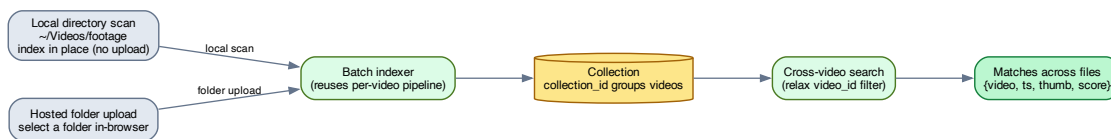


Figure 6: V2 Library Mode — both ingestion modes (local scan first, hosted upload second) feed one batch indexer; a collection scopes cross-video search.

- **Cross-video search.** Frames already carry `video_id` in ChromaDB, so this is mostly relaxing the per-video where filter and carrying the source video into each result. The expensive part (per-video indexing) already exists.
- **Collections.** A `collection_id` grouping scopes a query to “this folder.”
- **Two ingestion modes — both committed, local first:**
 - *Local directory scan* (self-hosted) — **implemented** (`v2-library-mode` branch): `POST /api/library/scan` walks a folder, indexes each video in place (nothing uploaded); the UI shows live per-video progress and searches across all of them. Frontend folder panel appears only on localhost.
 - *Hosted folder upload* — **implemented**: select a folder in the browser (`webkitdirectory`); the client uploads each video into a new collection (`POST /api/library/create` then `/api/library/{id}/upload`) and searches across them. Reuses the same cross-video engine. Per-file tier caps apply.
- **Constraint.** Indexing is CPU-bound (CLIP on CPU); first index of a large folder takes time — durable state + a progress UI make that acceptable.

Status: both ingestion modes are built and verified, and merged to main / deployed. Local scan = in-place (no upload, localhost only); hosted upload works anywhere. Caveat: uploaded collections live on Railway’s ephemeral disk, so they don’t survive a redeploy yet — the durable-storage foundation closes that gap.

Deferred: clip export + share links, YouTube ingest (legal), search-quality (hybrid ranking), React rewrite, face recognition (legal review), native mobile, GPU inference.

9 Known limitations (V1)

- **5-second frame granularity.** Moments shorter than the interval can be missed.
- **Single-process / single-node.** No horizontal scaling; CLIP runs on CPU.
- **Storage is one local volume.** Durable across restarts (DB + persistent volume), but not object storage / multi-node — fine for a single Railway service.
- **No clip export, no YouTube ingest, no face recognition** (deferred by spec).

10 About this document

The `.md` is the editable **source** (plain text — easy to diff, version, and edit precisely). The `.pdf` is the **rendered output** you read and share. Edit the Markdown (or a `diagrams/*.dot`), run `./docs/render.sh`, and `pandoc + xelatex` rebuild the PDF with freshly rendered Graphviz diagrams.

It is kept current as the project evolves (not mechanically every prompt) and pushed to GitHub whenever it changes. The change log records each meaningful update.

11 Change log

Date	Trigger	Change to this doc
2026-05-22	Initial request: PDF on the architecture.	Created the architecture document and rendered it to PDF.
2026-05-22	Auth0 tenant friendly-name fix.	No architecture change (cosmetic Auth0 dashboard setting).
2026-05-22	Auth0 dev-keys alert cleared.	No architecture change (own Google OAuth keys in Auth0).
2026-05-22	“what is v2 for this weekend”	Added the V2 roadmap (foundation + marquee).
2026-05-22	“use scrubless on a directory full of videos”	Set V2 marquee to Library Mode (cross-video search + batch ingestion).
2026-05-22	“do both, local first; commit v1 totally”	Locked both ingestion modes (local first); committed docs/, tagged v1.0.
2026-05-22	“fix structure; make it diagrammatic; push on update”	Restructured the document; added six Graphviz vector diagrams; switched cadence to “on meaningful change” + auto-push; documented the <code>.md/.pdf</code> split.
2026-05-22	“proceed” (build V2)	Implemented Library Mode — local directory scan + cross-video search (backend + UI) on <code>v2-library-mode</code> . Added the new endpoints to the API table, <code>collection_id/COLLECTIONS</code> to the data model + diagram, and marked local scan implemented in the roadmap.
2026-05-22	“isn’t it better to select the folder in Finder?”	Added a native folder picker (<code>POST /api/library/pick</code> via <code>osascript</code>) + a Browse button, since browsers can’t expose a folder’s absolute path to JS. Text-path input kept as a fallback.

Date	Trigger	Change to this doc
2026-05-22	"1 then 2" (merge+deploy, then phase 2)	Merged Library Mode to main (deployed). Built phase 2 — hosted folder upload (/api/library/create + /api/library/{id}/upload, webkitdirectory UI); both ingestion modes now implemented. Updated the V2 diagram + roadmap.
2026-05-22	"yes tackle it now" (durable storage)	Durable storage foundation: persisted videos/collections to the DB + restore_state() on boot (resumes interrupted indexing; idempotent re-index). Library now survives restart — verified locally. Requires a Railway volume at /app/storage + DATABASE_URL. Rewrote the data-model diagram; updated deployment + limitations + roadmap.
2026-05-22	"verify persistence"	Verified durable storage on production across a real redeploy: collection metadata (Postgres), embeddings + frames + source files (volume) all survived with no re-upload; delete path confirmed on prod. Volume + DATABASE_URL are correctly configured.